МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Ижевский государственный технический университет имени М. Т. Калашникова»

Факультет «Информационные технологии»

Кафедра «Программное обеспечение»

Отчет по лабораторной работе №1

по дисциплине «Сети и телекоммуникации»

Вариант 13

Выполнил

студент гр. Б22-191-1 Малышев В.И.

Принял

к. т. н., доцент Корнилов И.Г.

Ижевск 2026

Оглавление

1. ЦЕЛЬ РАБОТЫ 3
2. ПОСТАВЛЕННАЯ ЗАДАЧА 3
Индивидуальное задание 4
3. ХОД РАБОТЫ 5
3.1. Теоретические сведение 5
3.2. Описание программного кода 6
3.2.1. Клиент (client.py): 6
3.2.2. Сервер (server.py) 10
3.3. Порядок работы с приложением 16
3.4. Пример работы программы 19
ЗАКЛЮЧЕНИЕ 21

1. ЦЕЛЬ РАБОТЫ

Изучить базовые принципы клиент-серверных приложений.

2. ПОСТАВЛЕННАЯ ЗАДАЧА

Разработать простейшее клиент-серверное приложение, работающее по протоколу TCP. При создании приложения необходимо учитывать следующее:

1. Имеется клиент (клиентская программа), которой при старте задаются следующие параметры: IP-адрес и номер TCP-порта сервера.

2. Имеется сервер (серверная программа), которой при старте задается номер обслуживаемого TCP-порта.

3. Работа клиента заключается в отправке серверу текстовой посылки, набранной на клавиатуре, и отображении полученного ответа от сервера.

4. Работа сервера заключается в принятии текстовой посылки от клиента и отправки клиенту ответа. Формирование текста ответа сервера должно осуществляться путем отправки полученного от клиента сообщения и если в нем есть ключевая фраза, то преобразованием полученной после ключевой фразы текстовой посылки в соответствии с индивидуальным вариантом задания и дополнительной отправкой преобразованного сообщения.

5. если клиент не смог связаться с сервером - он уведомляет об этом пользователя и завершает работу.

6. если сервер обнаружил, что назначенный порт занят - он уведомляет об этом пользователя и завершает работу.

Работу рекомендуется выполнять на языке программирования Python, но допускается выполнение работы на других языках программирования (C#, C++, JAVA).

- Реализовать циклическое получение, отправку и отображение сообщений.
- Возможность работы нескольких клиентов одновременно с передачей им всем одинаковой информации.
- Добавить ввод имени клиента.
- Реализовать отправку сервером списка подключенных клиентов.
- Обеспечить выбор пользователем клиентов для включения в группу и отображение сообщений только клиентов из группы.
- Отправка сервером клиентскому приложению сообщений только нужных пользователей.
- При выводе сообщения показывать имя клиента, от которого оно пришло.
- Реализовать графический интерфейс у клиентского приложения.

Индивидуальное задание

Все символы русского алфавита передаются как принятый символ в верхнем регистре и 3 символа русского алфавита, предшествующих принятому символу, в нижнем регистре.

3. ХОД РАБОТЫ

3.1. Теоретические сведение

Сокет — программный интерфейс для обмена данными между процессами в сети или на одной машине. В модели клиент-сервер:

- Сервер создаёт сокет, привязывает его к IP-адресу и порту (bind), переходит в режим прослушивания (listen) и ожидает подключений (accept).
- Клиент создаёт сокет и инициирует соединение с сервером (connect).

В работе используется:

- AF_INET — семейство адресов IPv4.
- SOCK_STREAM — потоковый сокет поверх протокола TCP (гарантирует надёжную доставку данных).

3.2. Описание программного кода

3.2.1. Клиент (client.py):

Текст программы:

```
import socket
import threading
import tkinter as tk
from tkinter import scrolledtext

HOST = '127.0.0.13'
PORT = 8888


class ChatClient:
    def __init__(self, root):
        self.root = root
        self.root.title("cleint")

        self.socket = None
        self.buffer = ""
        self.my_group = set()

        self.chat = scrolledtext.ScrolledText(root, state='disabled',
height=20)
        self.chat.pack(fill="both", padx=10, pady=5)

        frame = tk.Frame(root)
        frame.pack(fill="x")

        tk.Label(frame, text="Пользователи (клик):").pack(anchor="w")

        self.users = tk.Listbox(frame, selectmode=tk.MULTIPLE, height=5)
        self.users.pack(fill="x")

        self.users.bind("<<ListboxSelect>>", self.update_group_from_ui)

        self.group_label = tk.Label(root, text="Общий чат", fg="blue")
        self.group_label.pack()

        bottom = tk.Frame(root)
        bottom.pack(fill="x", pady=5)

        self.entry = tk.Entry(bottom)
        self.entry.pack(side="left", fill="x", expand=True, padx=5)
        self.entry.bind("<Return>", self.send)

        tk.Button(bottom, text="➤", command=self.send).pack(side="right")

        self.connect()

    def write(self, msg):
        self.chat.config(state='normal')
        self.chat.insert(tk.END, msg + "\n")
        self.chat.config(state='disabled')
        self.chat.yview(tk.END)

    def connect(self):
        name = self.ask_name()
```

```
        try:
            self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.socket.connect((HOST, PORT))
            self.socket.sendall((name + "\n").encode())

            threading.Thread(target=self.receive, daemon=True).start()
            self.write("Подключено")

        except:
            self.write("Ошибка подключения")

    def ask_name(self):
        popup = tk.Toplevel(self.root)
        popup.title("Имя")

        tk.Label(popup, text="Введите имя").pack()
        entry = tk.Entry(popup)
        entry.pack()

        result = {"name": "Аноним"}

        def ok():
            result["name"] = entry.get().strip() or "Аноним"
            popup.destroy()

        tk.Button(popup, text="OK", command=ok).pack()
        self.root.wait_window(popup)

        return result["name"]

    def update_group_from_ui(self, event=None):
        selected = [self.users.get(i) for i in self.users.curselection()]
        self.my_group = set(selected)

        if self.my_group:
            self.group_label.config(text="Группа: " + ",
".join(self.my_group))
        else:
            self.group_label.config(text="Общий чат")

        try:
            if self.my_group:
                msg = "/group " + ",".join(self.my_group)
            else:
                msg = "/group"

            self.socket.sendall((msg + "\n").encode())
        except:
            pass

    def receive(self):
        try:
            while True:
                data = self.socket.recv(1024)
                if not data:
                    break

                self.buffer += data.decode()

                while "\n" in self.buffer:
                    msg, self.buffer = self.buffer.split("\n", 1)
                    self.process(msg)

        except:
```

```
            self.write("Соединение потеряно")

    def process(self, msg):
        if msg.startswith("SYS:USERS:"):
            users = msg.split(":", 2)[2].split(",")

            self.users.delete(0, tk.END)
            for u in users:
                if u:
                    self.users.insert(tk.END, u)

        elif msg.startswith("SYS:MSG:"):
            self.write("[СИСТЕМА] " + msg.split(":", 2)[2])

        elif msg.startswith("MSG:"):
            parts = msg.split(":", 2)
            if len(parts) >= 3:
                sender = parts[1]
                text = parts[2]

                if self.my_group and sender not in self.my_group:
                    return

                self.write(f"{sender}: {text}")

    def send(self, event=None):
        msg = self.entry.get().strip()
        if not msg:
            return

        try:
            self.socket.sendall((msg + "\n").encode())

            if not msg.startswith("/"):
                if self.my_group:
                    self.write(f"Вы → {', '.join(self.my_group)}: {msg}")
                else:
                    self.write(f"Вы → ВСЕ: {msg}")

        except:
            self.write("Ошибка отправки")

        self.entry.delete(0, tk.END)


if __name__ == "__main__":
    root = tk.Tk()
    app = ChatClient(root)
    root.mainloop()
```

Логика работы:

- Клиент создаёт TCP-сокет с использованием библиотеки socket.
- Выполняется подключение к серверу по адресу 127.0.0.1:8888.
- При запуске пользователю предлагается ввести имя, которое отправляется серверу.

- После подключения запускается отдельный поток для приёма сообщений от сервера.
- В графическом интерфейсе реализованы:
  - поле отображения сообщений чата;
  - список подключённых пользователей;
  - поле ввода сообщений;
  - кнопка отправки сообщения;
  - отображение текущей выбранной группы.
- При выборе пользователей автоматически формируется команда /group, которая отправляется серверу.
- Пользователь вводит сообщение:
  - сообщение отправляется серверу в кодировке UTF-8;
  - если это обычное сообщение, оно отображается в окне чата с указанием получателей:
    - «Вы → ВСЕ» — если выбран общий чат;
    - «Вы → Имя1, Имя2» — если выбрана группа.
- В отдельном потоке клиент:
  - принимает данные от сервера;
  - накапливает их в буфере;
  - разделяет сообщения по символу \n.
- Обработка сообщений:
  - SYS:USERS — обновление списка пользователей;
  - SYS:MSG — вывод системных сообщений;
  - MSG — отображение сообщений других клиентов.
- Если выбрана группа, клиент:
  - отображает только сообщения от пользователей из этой группы.
- При разрыве соединения клиент уведомляет пользователя и завершает работу.

3.2.2. Сервер (server.py)

Текст проргаммы:

```
import socket
import threading

host = '127.0.0.6'
port = 8888

russian_alphabet = "абвгдежзийклмнопрстуфхцчшщъыьэюя"

clients = {}
clients_lock = threading.Lock()


def transform_text(text):
    result = ""
    for char in text:
        lower_char = char.lower()

        if lower_char in russian_alphabet:
            index = russian_alphabet.index(lower_char)

            prev_chars = []
            for i in range(3, 0, -1):
                if index - i >= 0:
                    prev_chars.append(russian_alphabet[index - i])

            result += char.upper() + "".join(prev_chars)
        else:
            result += char

    return result


def send_user_list():
    with clients_lock:
        names = [data['name'] for data in clients.values()]
        connections = list(clients.keys())

    msg = f"SYS:USERS:{','.join(names)}\n"

    for conn in connections:
        try:
            conn.sendall(msg.encode('utf-8'))
        except:
            pass


def broadcast_system(message, sender_conn=None):
    with clients_lock:
        connections = list(clients.keys())

    for conn in connections:
        if conn == sender_conn:
            continue
        try:
            conn.sendall(f"SYS:MSG:{message}\n".encode('utf-8'))
        except:
            pass


def handle_client(conn, addr):
```

```
    buffer = ""
    client_name = None

    try:
        name_data = conn.recv(1024)
        if not name_data:
            return

        client_name = name_data.decode('utf-8').strip()

        with clients_lock:
            clients[conn] = {
                'name': client_name,
                'group': set()
            }

        print(f"[+] {client_name} подключился")

        send_user_list()

        conn.sendall(f"SYS:MSG:Добро пожаловать,
{client_name}!\n".encode('utf-8'))

        broadcast_system(f"{client_name} вошёл в чат", conn)

        while True:
            data = conn.recv(1024)
            if not data:
                break

            buffer += data.decode('utf-8')

            while "\n" in buffer:
                message, buffer = buffer.split("\n", 1)
                message = message.strip()

                if not message:
                    continue

                if message.startswith("/"):
                    parts = message.split(" ", 1)
                    cmd = parts[0]

                    if cmd == "/group":
                        if len(parts) > 1:
                            names = [n.strip() for n in parts[1].split(",")
if n.strip()]
                            with clients_lock:
                                clients[conn]['group'] = set(names)

                            conn.sendall(f"SYS:MSG:Группа: {',
'.join(names)}\n".encode('utf-8'))
                        else:
                            with clients_lock:
                                clients[conn]['group'] = set()
                            conn.sendall("SYS:MSG:Группа
сброшена\n".encode('utf-8'))

                    continue

                print(f"[{client_name}]: {message}")

                main_message = message
                extra_message = None
```

```
                if "<@>" in message:
                    parts = message.split("<@>", 1)
                    if len(parts) > 1:
                        text_to_transform = parts[1].strip()
                        if text_to_transform:
                            extra_message = transform_text(text_to_transform)

                with clients_lock:
                    sender_group = clients[conn]['group']
                    clients_copy = dict(clients)

                for c_conn, c_data in clients_copy.items():
                    if c_conn == conn:
                        continue

                    if sender_group and c_data['name'] not in sender_group:
                        continue

                    try:
                        msg = f"MSG:{client_name}:{main_message}\n"
                        c_conn.sendall(msg.encode('utf-8'))

                        if extra_message:
                            msg2 = f"MSG:{client_name}:[ПРЕОБРАЗОВАНО]:
{extra_message}\n"
                            c_conn.sendall(msg2.encode('utf-8'))

                    except:
                        pass

                if extra_message:
                    conn.sendall(f"SYS:MSG:Преобразовано:
{extra_message}\n".encode('utf-8'))

    except Exception as e:
        print(f"[!] Ошибка: {e}")

    finally:
        with clients_lock:
            if conn in clients:
                name = clients[conn]['name']
                del clients[conn]
                print(f"[-] {name} отключился")

        conn.close()
        send_user_list()
        broadcast_system(f"{client_name} вышел")


def main():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        s.bind((host, port))
    except OSError:
        print("Ошибка: порт занят!")
        return

    s.listen()
    print(f"Сервер запущен {host}:{port}")

    try:
        while True:
```

```
            conn, addr = s.accept()
            threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()
    except KeyboardInterrupt:
        print("Остановка сервера")
    finally:
        s.close()


if __name__ == "__main__":
    main()
```

Логика работы:

Инициализация сервера:

• Сервер создаёт TCP-сокет (AF_INET, SOCK_STREAM).

• Выполняется привязка к адресу 127.0.0.1:8888.

• Если порт занят — выводится сообщение об ошибке, и программа завершается.

• Сервер переходит в режим прослушивания входящих подключений.

• В бесконечном цикле принимаются новые подключения клиентов.

• Для каждого клиента создаётся отдельный поток обработки.

Регистрация клиента:

• Сервер получает имя клиента (первое сообщение после подключения).

• Данные клиента сохраняются в словаре: имя и персональная группа получателей.

• Всем подключённым клиентам отправляется обновлённый список пользователей.

• Новому клиенту отправляется приветствие и справка по командам.

• Остальным участникам рассылается уведомление о входе нового пользователя.

Приём и обработка сообщений:

• Сервер принимает данные от клиента порциями по 1024 байт.

• Данные накапливаются в буфере и разделяются по символу "\n".

• Каждое сообщение декодируется из UTF-8 и очищается от пробелов.

• Пустые строки игнорируются.

Преобразование текста:

• Если сообщение содержит маркер "<@>", текст после него извлекается.

• Для каждой русской буквы выполняется преобразование:

- буква переводится в верхний регистр;

- добавляются три предыдущие буквы алфавита в нижнем регистре.

• Результат отправляется как дополнительное сообщение с пометкой.

Маршрутизация сообщений:

• Сервер проверяет группу отправителя из словаря клиентов.

• Для каждого получателя проверяется фильтрация:

- если группа отправителя пуста — сообщение рассылается всем;

- если группа установлена — сообщение получают только участники группы.

• Сообщение отправляется в формате "MSG:Имя:Текст".

• Отправителю подтверждается доставка и результат преобразования.

Отключение клиента:

• При получении пустого пакета соединение считается разорванным.

• Клиент удаляется из общего словаря активных пользователей.

• Всем остальным рассылается обновлённый список участников.

• Участникам отправляется уведомление о выходе клиента.

• Соединение закрывается, поток завершает работу.

Завершение работы сервера:

• При нажатии Ctrl+C цикл приёма подключений прерывается.

- Основной сокет закрывается.
- Программа завершает работу.

3.3. Порядок работы с приложением

1) Запуск сервера

В терминале запускается серверная программа.

- Сервер успешно привязывается к адресу 127.0.0.1:8888.
- Переходит в режим ожидания подключения клиентов.
- В консоли отображается сообщение о запуске сервера.

Рис. 1 – запуск сервера

2) Запуск клиента

В отдельном окне запускается клиентская программа с графическим интерфейсом.

- Пользователь вводит имя.
- Устанавливается соединение с сервером.
- Открывается окно чата.

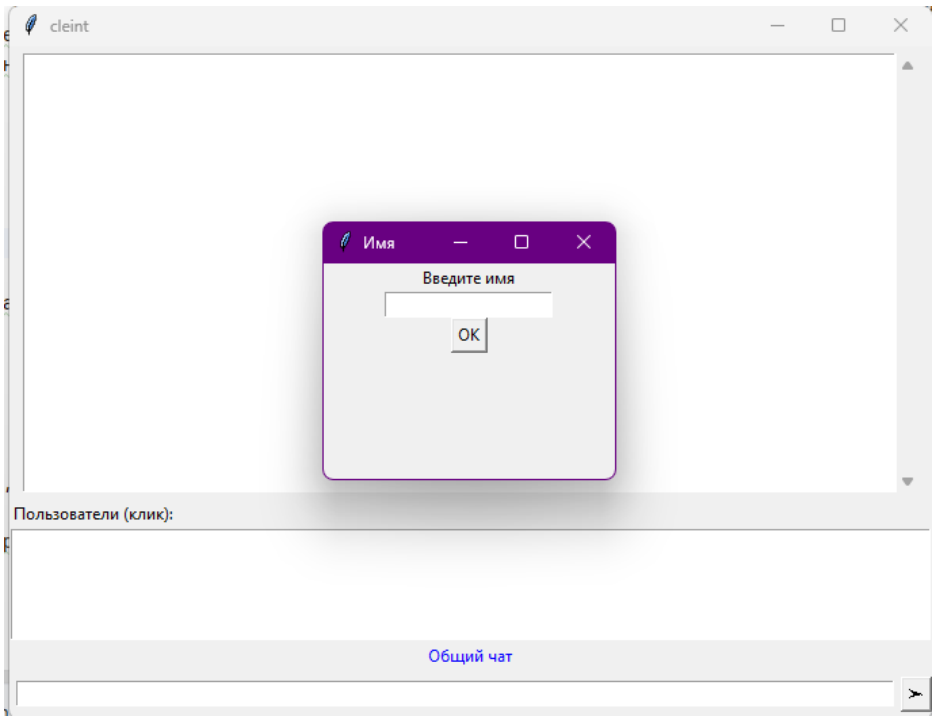

Рис. 2 – Ввод имени для пользователя

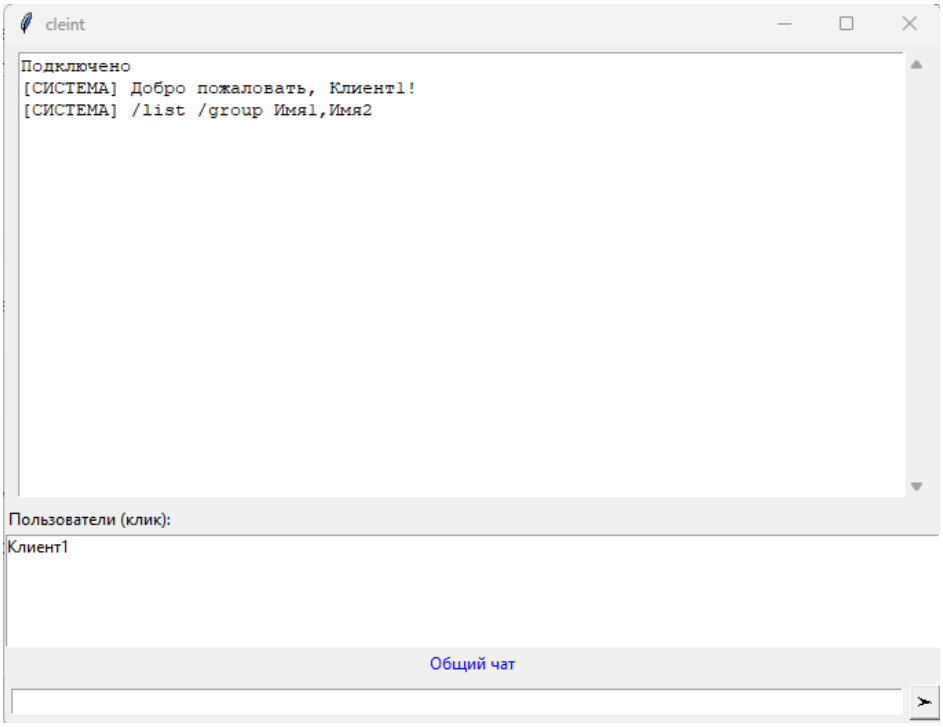

Рис. 3 – Окно чата

3) Работа клиента

- В окне отображаются:
  - список пользователей;
  - сообщения;
  - текущая группа.

- Пользователь может:
    - выбрать группу (через список пользователей);
    - отправить сообщение.

4) Обмен сообщениями

- Клиент отправляет сообщение серверу.
- Сервер:
    - принимает сообщение;
    - при необходимости выполняет преобразование;
    - отправляет:
        - исходное сообщение;
        - преобразованное сообщение (если есть <@>).
- Клиенты получают и отображают сообщения.

5) Завершение работы

- При закрытии клиента:
    - соединение разрывается;
    - сервер удаляет клиента из списка.
- Сервер продолжает работу с другими клиентами.

3.4. Пример работы программы

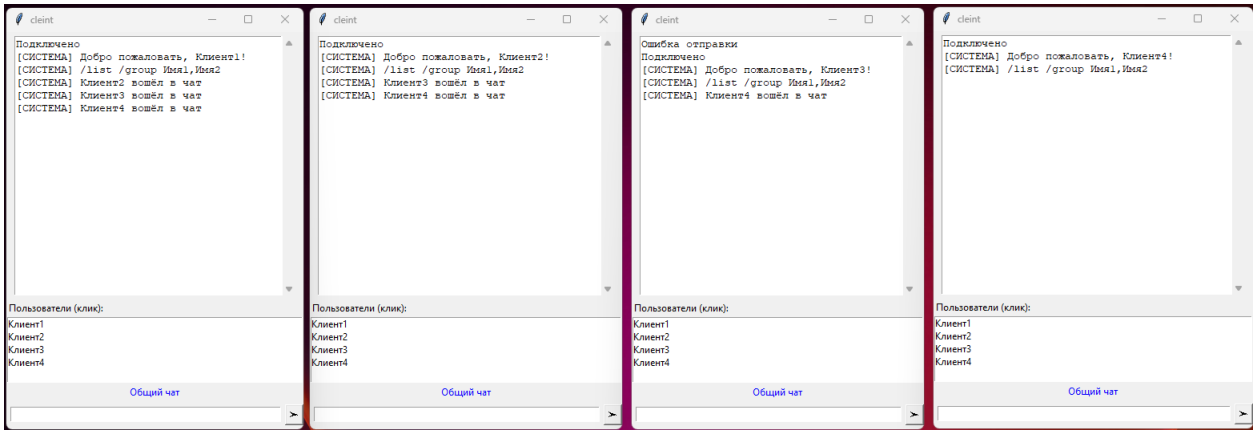

Рис. 4 – Запуск четырех клиентов

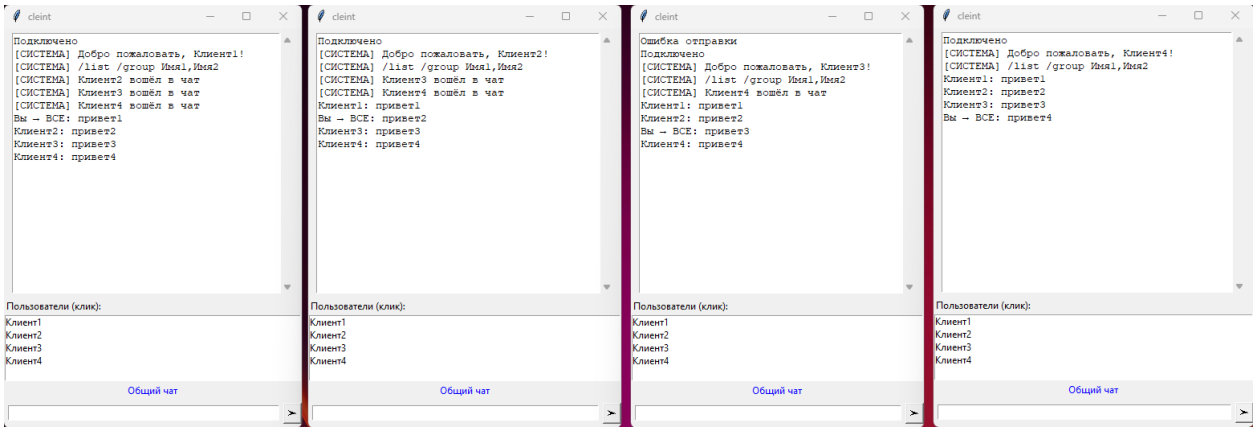

Рис. 5 – Отправка текстового сообщения

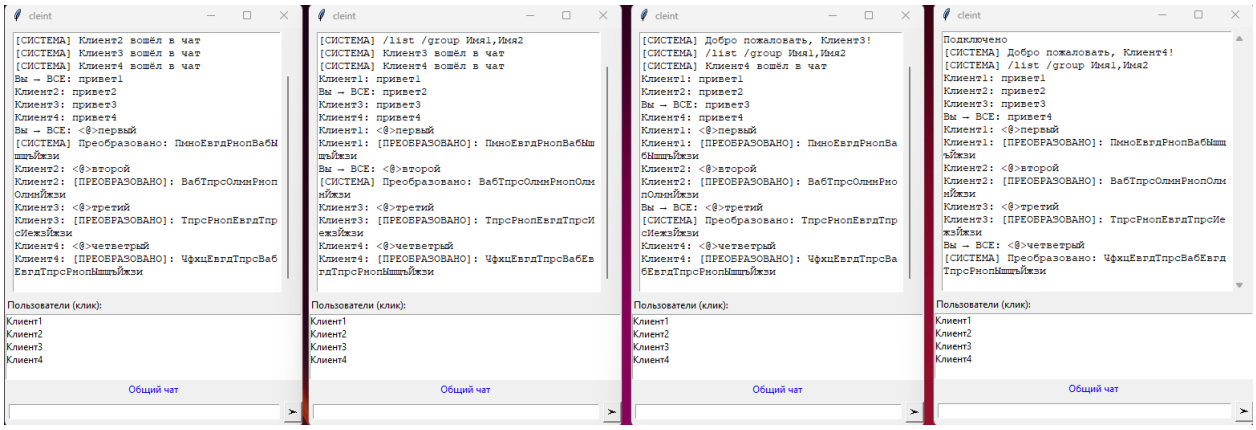

Рис. 6 – Отправка специального текстового сообщения

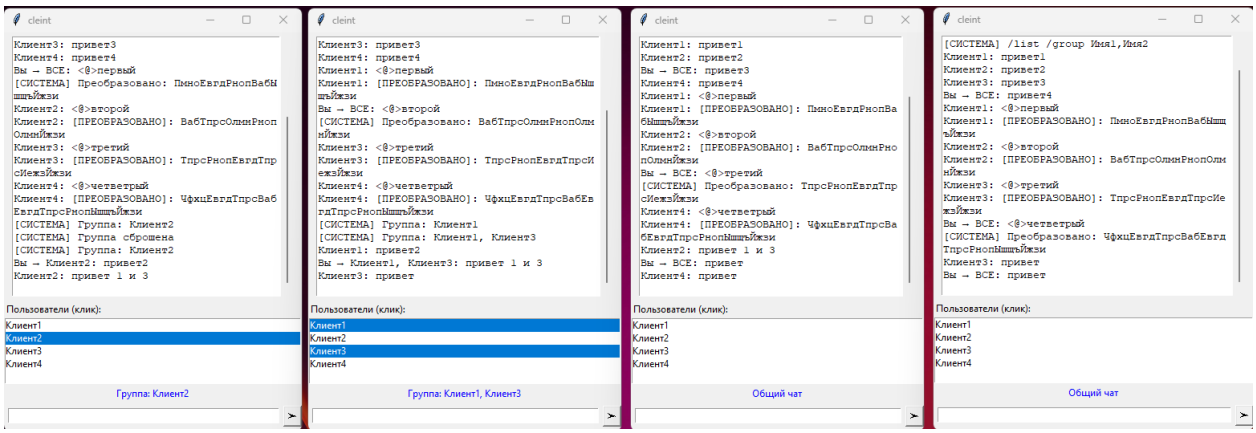

Рис. 7 – Выбор группы пользователей

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы разработано многопользовательское клиент-серверное приложение на протоколе TCP с графическим интерфейсом. Реализованы подключение нескольких клиентов, фильтрация сообщений по группам и преобразование текста по маркеру <@>. Сервер корректно обрабатывает соединения и рассылку данных. Все требования задания выполнены полностью, программа функционирует стабильно.